

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена Трудового Красного Знамени федеральное государственное
бюджетное образовательное учреждение высшего образования
«Московский технический университет связи и информатики»

Кафедра «Информатика»

Лабораторная работа №5
**«Разработка алгоритмов решения сложных задач
методом пошаговой детализации и их
программная реализация»**
по Разделу
**«Разработка алгоритмов решения сложных задач
методом пошаговой детализации и их
программная реализация»**
по дисциплине
«Введение в информационные технологии»

Вариант 1

Выполнил: студент гр. ЗРС2502 Баранова Е.В

Проверил: Рюмин Ю.А.

Москва, 2025 г.

1) Общее задание

- 1) Изучить основные положения технологий структурного проектирования алгоритмов и метода пошаговой детализации, средства MS Visio для графической визуализации алгоритмов.
- 2) Выбрать вариант индивидуального задания из таблицы 5.1.
- 3) Проанализировать постановку задачи своего варианта индивидуального задания и, если необходимо, уточнить ее у преподавателя.
- 4) Провести формализацию решения задачи. Определить этапы ее решения с использованием метода пошаговой детализации.
- 5) Разработать схемы алгоритмов процедур на различных этапах метода пошаговой детализации и схему иерархии процедур.
- 6) Утвердить у преподавателя результаты выполнения п.п. 4-5.
- 7) Разработать программный код функций VC++ по алгоритмам п. 5
- 8) Создать консольный проект, содержащий три файла исходного кода: файл с главной функцией main, файл с функциями ввода и вывода данных и файл с функциями, решающими предписанные задачи. Обмен данными между функциями должен осуществляться через параметры и возвращаемые значения, без использования глобальных переменных. Главная функция main должна содержать только операторы вызова разработанных функций.
- 9) Подготовить варианты исходных данных для тестирования проекта.
- 10) Выполнить проект с тестовыми исходными данными и получить результаты. Проверить правильность результатов.
- 11) Оформить отчет по работе в среде MS Word. Изобразить все схемы, используя средства MS Visio.
- 12) Представить преподавателю отчет по работе.
- 13) Ответить на замечания преподавателя по выполненной работе и на заданные им вопросы по теме.
- 14) Получить отметку о выполнении и защите работы.

2) Индивидуальное задание

Определите периметры правильных n – угольников (10 – угольника, 50 – угольника, 100– угольника), вписанных в окружность заданного радиуса R . Сторона правильного n -угольника $a = 2 * R * \sin \frac{180^\circ}{n}$; периметр n -угольника $p = n * a$.

3) Формализация задания

Для универсальности программы количество сторон многоугольника a сделаем вводимой переменной наравне с радиусом R . Выводимый результат решения – периметр, имеющий наименование P . Для вычисления будем также использовать переменную a , обозначающую длину стороны.

4) Разработка алгоритмов решения задач

Начнем проектирование алгоритма методом «сверху вниз» .

4.1) На самом верхнем (первом) уровне алгоритм решения задачи можно укрупненно представить в виде вызова главной процедуры с именем `main` (рисунок 1), где `Calc` (процедура) вычисления значений переменных a и P по заданному значению переменных n и R .

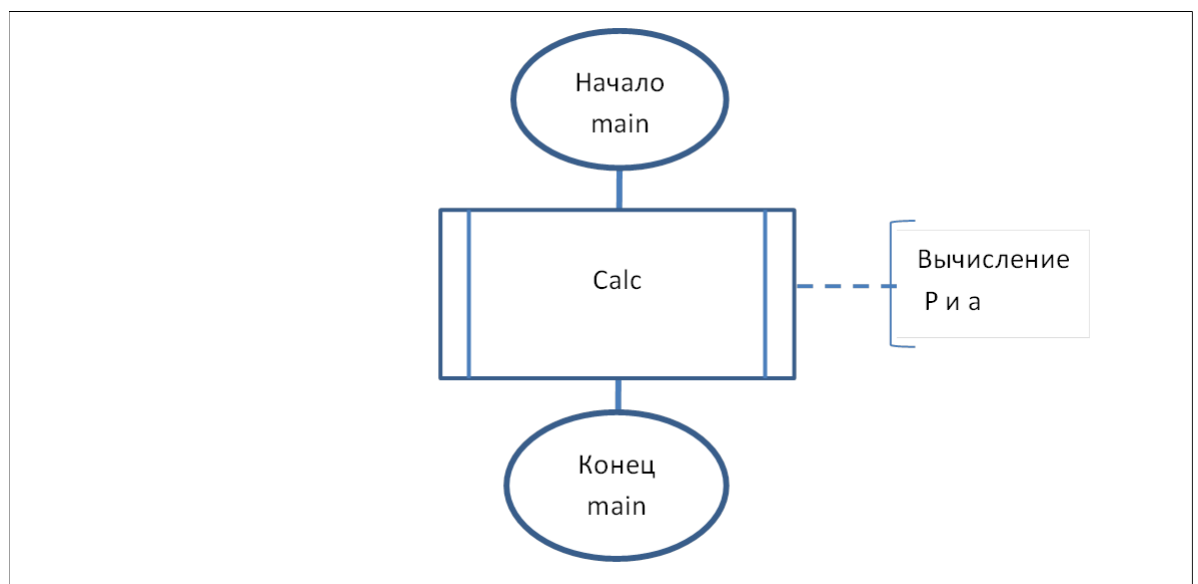


Рисунок 1. – Схема алгоритма главной процедуры `main` для всех проектов

4.2) На следующем, втором уровне, детализируем алгоритм процедуры Calc путем представления его в виде последовательности следующих трех процедур (рисунок 2):

- процедуры ввода значения x с именем GetR;
- процедуры вычисления y и z с именем CompP;
- процедуры вывода вычисленных значений периметра и площади с именем Put.

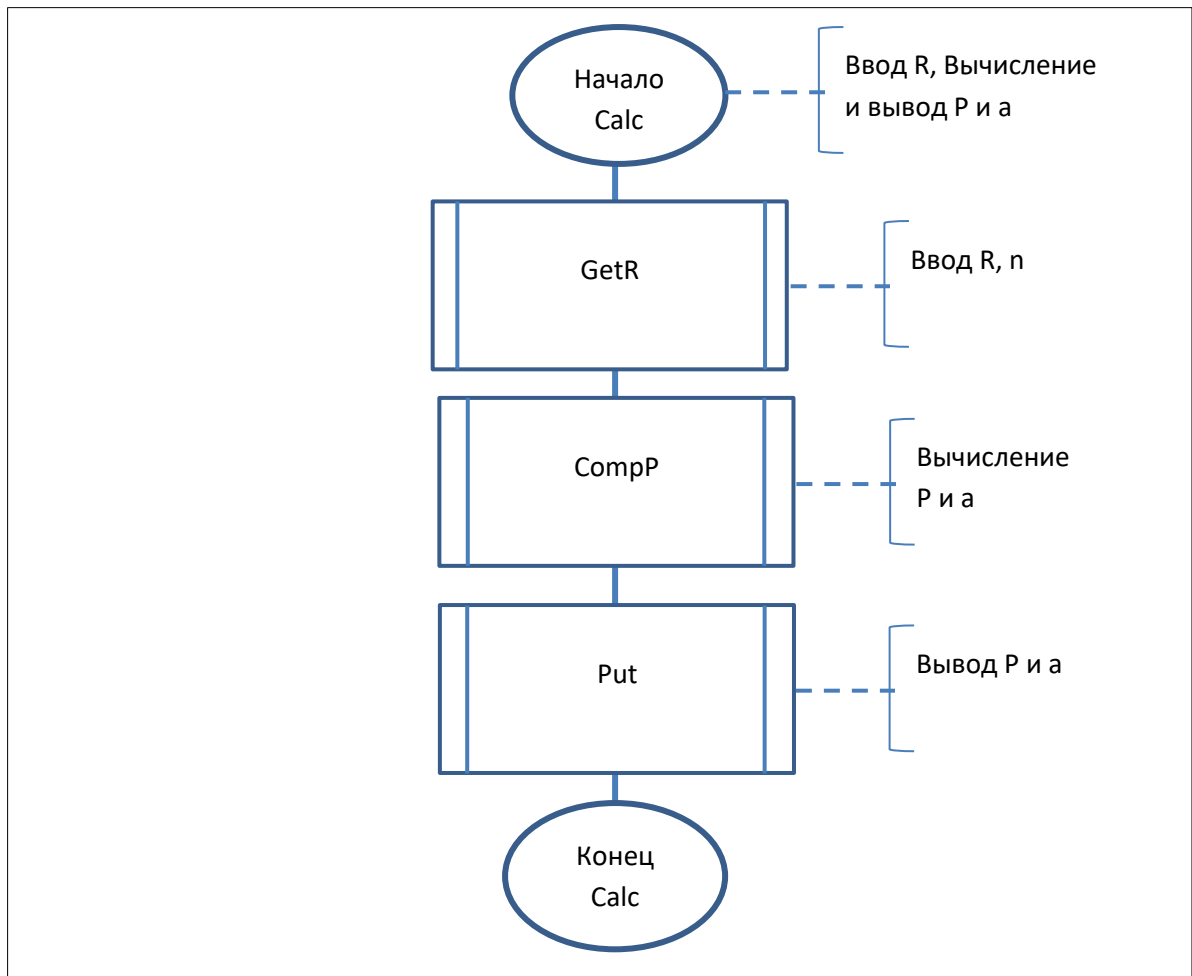


Рисунок 2. – результат второго уровня детализации алгоритма

4.3)Перейдем к следующему, третьему уровню детализации.

Процедуры GetR и Put дальнейшей детализации не требуют, так как средства ввода-вывода имеются в любом языке программирования. Поэтому на следующем, третьем шаге детализируем алгоритм процедуры вычисления P и a Comp (рисунок 3).

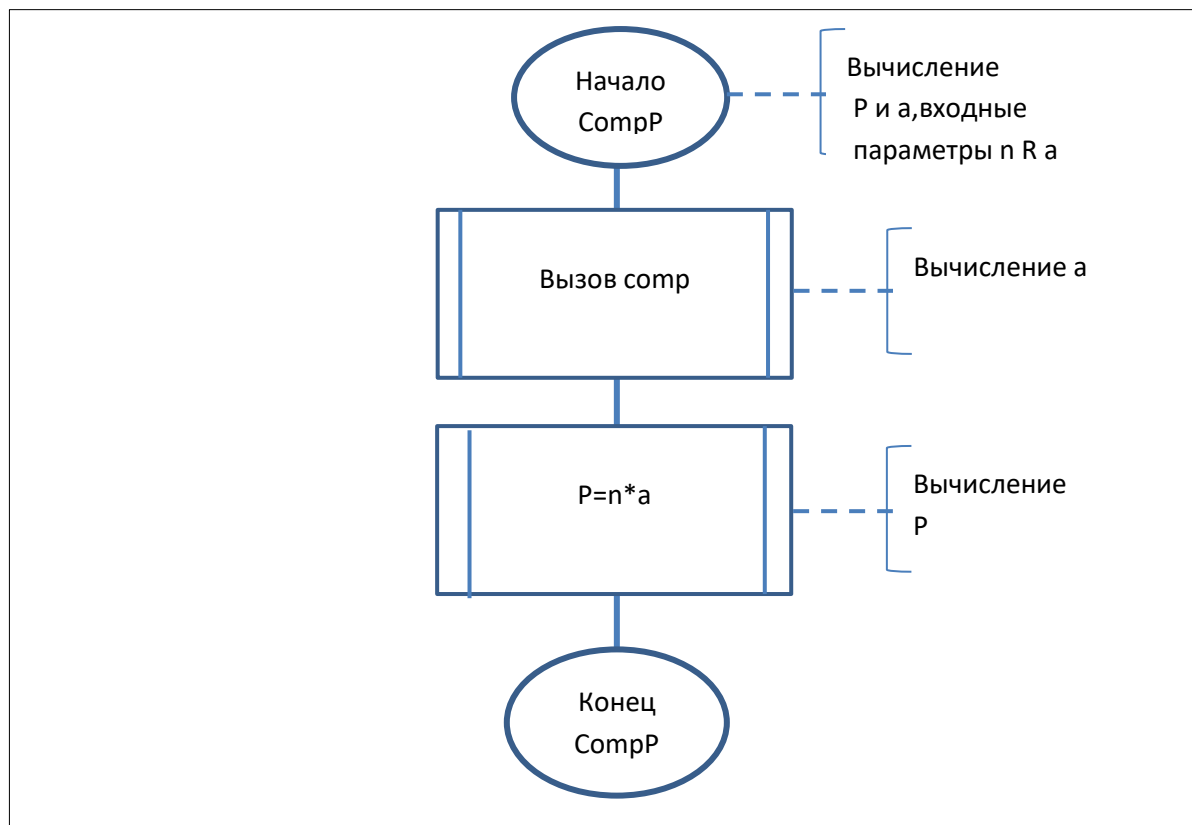


Рисунок 3 – Схема алгоритма процедуры comp

4.4) На последнем, четвертом уровне детализации разработаем схему алгоритма процедуры comp, вычисляющей значение a.(рисунок 4).

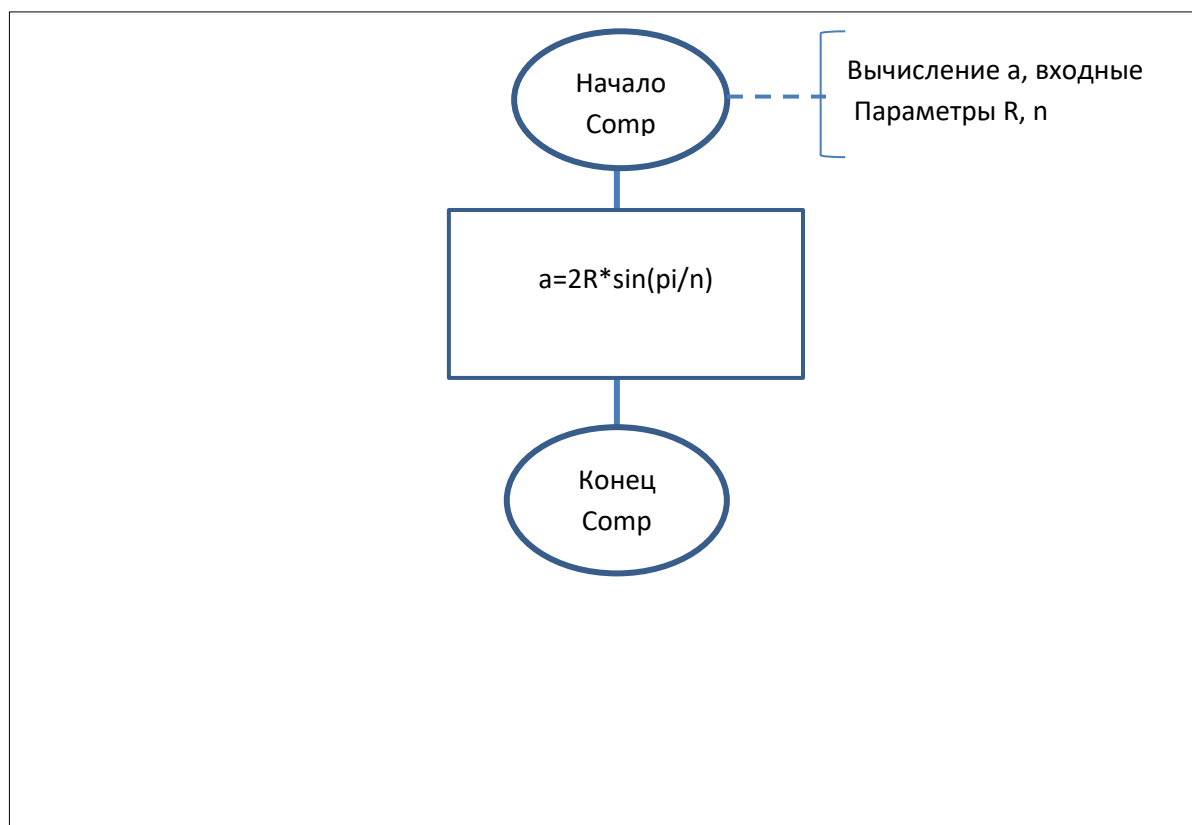


Рисунок 4 – схема алгоритма comp

Схема иерархии процедур для решения задачи изображена на рисунке 5.

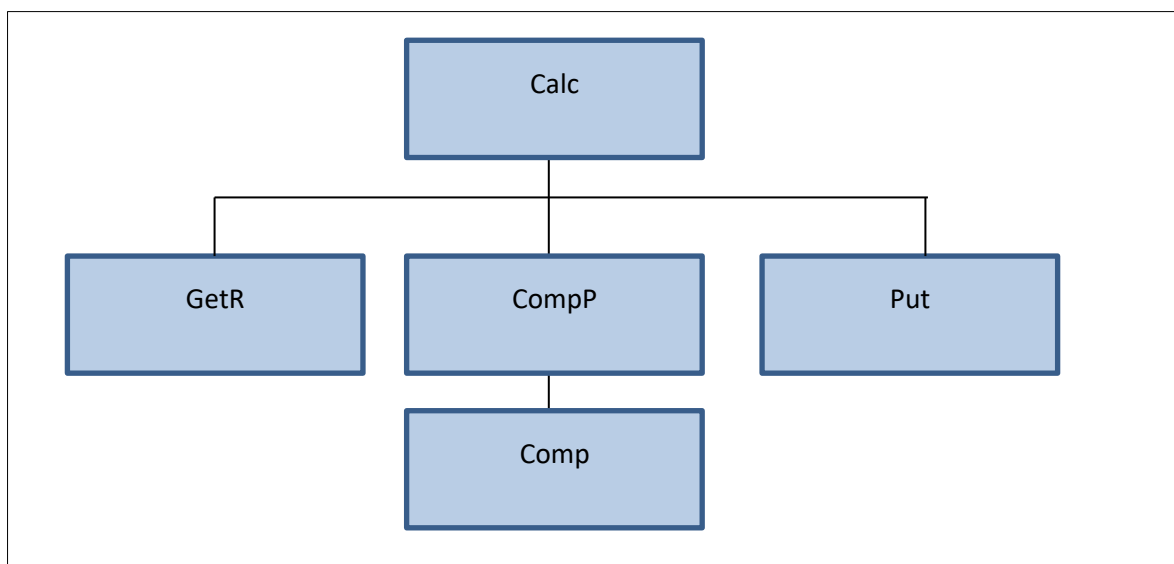


Рисунок 5 – Схема иерархии процедур для решения задачи

5) Разработка программного проекта

При разработке программного проекта внесем незначительное упрощение: избавимся от лишней “двухэтажности” и реализуем процедуру Calc в главной функции проекта main.

Для решения нашей задачи создадим проект, в котором будет три файла исходного кода: помимо файла с главной функцией main создадим файл с функциями GetR и Put для ввода и вывода и объединим в отдельный файл все остальные функции, участвующие в решении задачи.

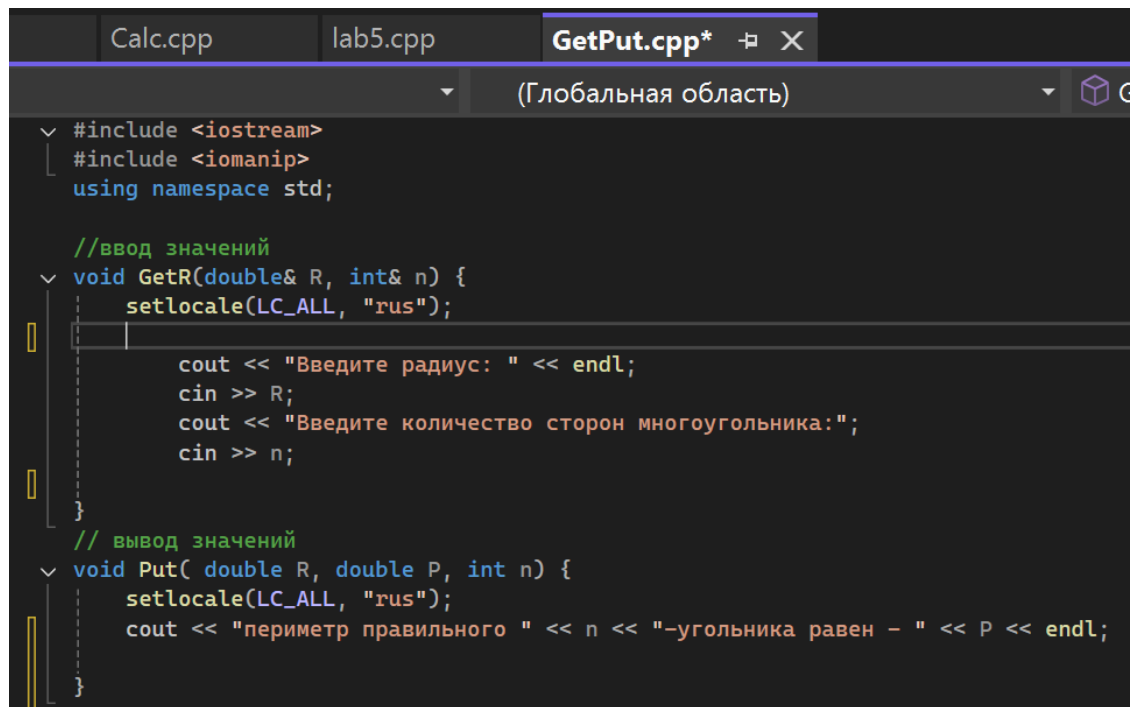
Программный код проекта представлен ниже на рисунках 6 – 8.

В первый файл с именем GetPut.cpp (рисунок 6) запишем функции ввода исходных данных GetR и вывода результатов Put. Обе эти функции типа void, так как не имеют возвращаемого значения. Функция ввода значений R и n GetR имеет два выходной параметр и ни одного входного параметра. Функция Put, предназначенная для вывода P, R и n, имеет 3 входных параметра и ни одного выходного параметра. Так как функции этого файла используют объекты cin и cout, то в нем присутствуют директивы #include<iostream> и using namespace std.

Во второй файл с именем Calc.cpp (рисунок 7) запишем функции, выполняющие необходимые вычисления для решения задачи. Функция CompP определена как функция с двумя входными параметрами R и n и

двумя выходными параметрами а и Р. Возвращаемого значения функция не имеет.

Функция Comp, вычисляющая а, имеет два входных параметров (значение R, n) и возвращаемое значение – вычисленную величину многочлена.



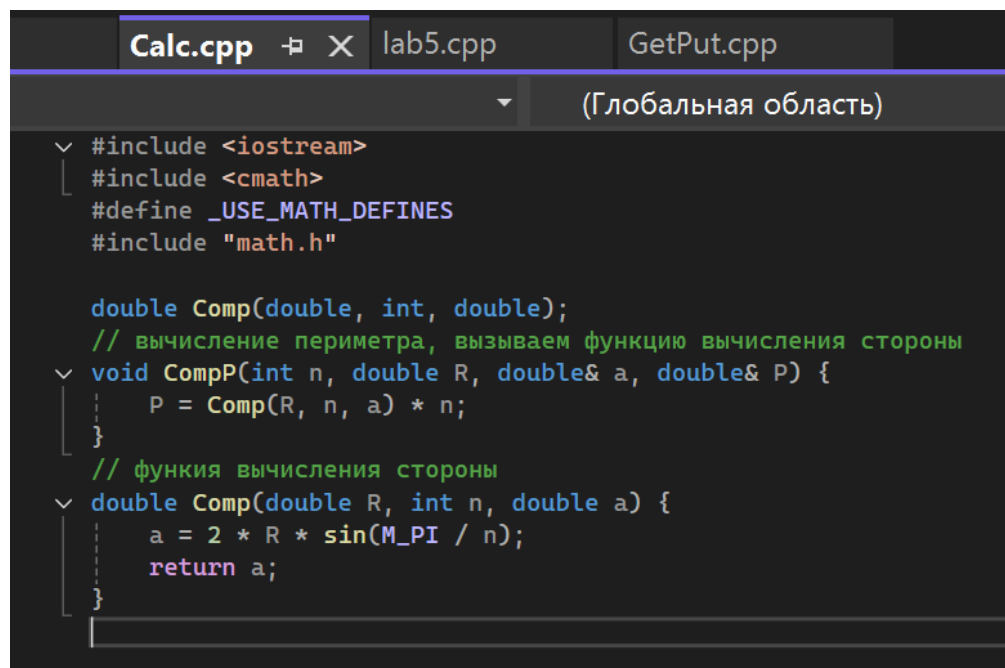
```
#include <iostream>
#include <iomanip>
using namespace std;

//ввод значений
void GetR(double& R, int& n) {
    setlocale(LC_ALL, "rus");

    cout << "Введите радиус: " << endl;
    cin >> R;
    cout << "Введите количество сторон многоугольника:";
    cin >> n;
}

// вывод значений
void Put( double R, double P, int n) {
    setlocale(LC_ALL, "rus");
    cout << "периметр правильного " << n << "-угольника равен - " << P << endl;
}
```

Рисунок 6 – Программный код файла с функциями GetR и Put



```
Calc.cpp lab5.cpp GetPut.cpp
(Глобальная область)

#include <iostream>
#include <cmath>
#define _USE_MATH_DEFINES
#include "math.h"

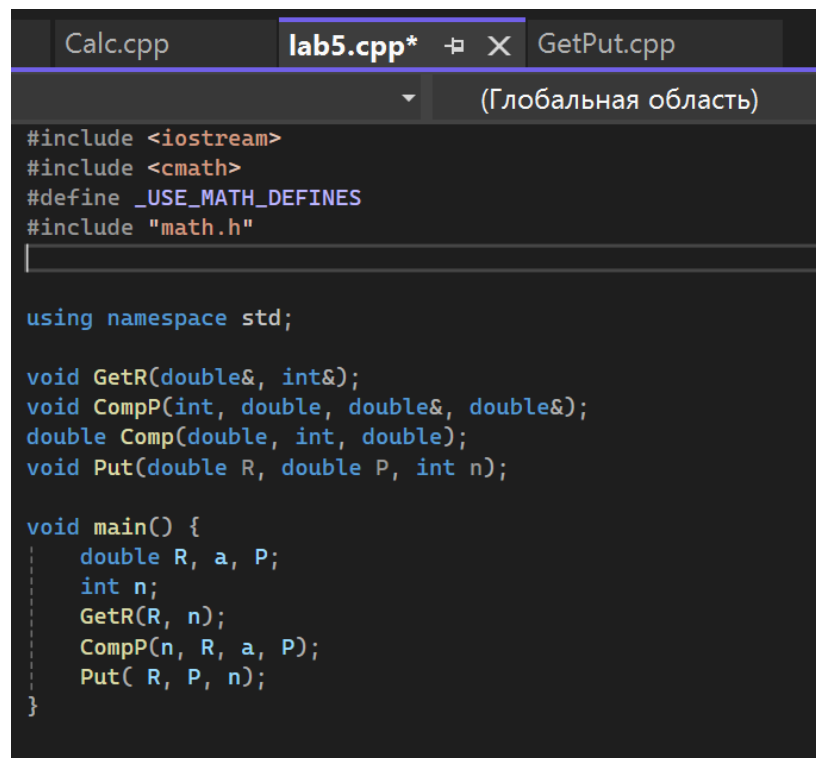
double Comp(double, int, double);
// вычисление периметра, вызываем функцию вычисления стороны
void CompP(int n, double R, double& a, double& P) {
    P = Comp(R, n, a) * n;
}

// функция вычисления стороны
double Comp(double R, int n, double a) {
    a = 2 * R * sin(M_PI / n);
    return a;
}
```

Рисунок 7 – Программный код файла с функциями CompP и Comp

Так как функция CompP определена перед вызываемой ею функцией Comp, то в начале файла записан прототип функции Comp.

В третьем файле с именем main.cpp (рисунок 8) находится главная функция main, программный код которой содержит только вызовы разработанных функций ввода исходных данных, решения задачи и вывода результатов. В этом файле присутствует директива препроцессора #include <iostream>, необходимая для команды задержки закрытия консольного окна system("PAUSE"). Перед определением главной функции main в файл включены прототипы вызываемых в ней функций GetR, CompP и Comp.



```
Calc.cpp lab5.cpp* GetPut.cpp
(Глобальная область)

#include <iostream>
#include <cmath>
#define _USE_MATH_DEFINES
#include "math.h"

using namespace std;

void GetR(double&, int&);
void CompP(int, double, double&, double&);
double Comp(double, int, double);
void Put(double R, double P, int n);

void main() {
    double R, a, P;
    int n;
    GetR(R, n);
    CompP(n, R, a, P);
    Put(R, P, n);
}
```

Рисунок 8 – Программный код файла с главной функцией main

6) Исходные данные для тестирования проекта

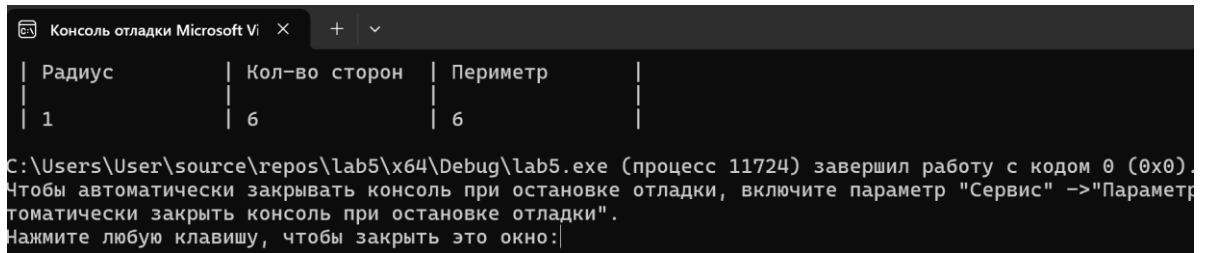
Протестируем проект при двух вариантах значений входных переменных R и n: R=1, n=6 и R=2, n=12.

Расчет на калькуляторе значения P при R=1, n=6 дает следующие результаты: a=1, P=6.

Расчет на калькуляторе значений P при R=2, n=12 дает следующие результаты: a=1,03528, P=12,42331.

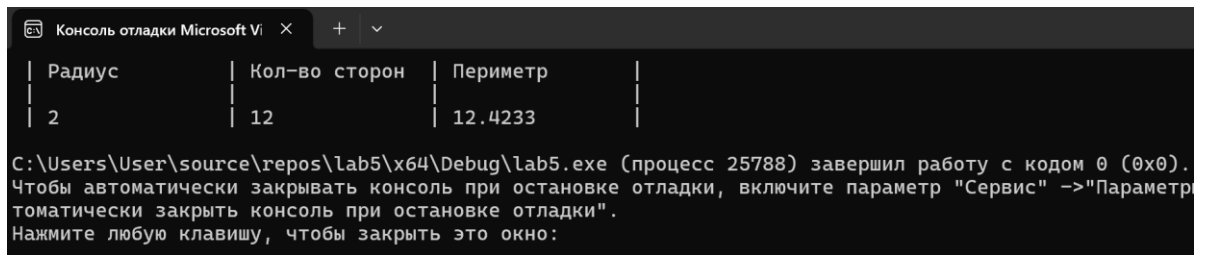
7) Результаты выполнения проекта

Результаты выполнения проекта при : $R=1$, $n=6$ приведены на рисунке 9, при $R=2$, $n=12$ – на рисунке 10.



```
Консоль отладки Microsoft Vi X + v
| Радиус | Кол-во сторон | Периметр |
| 1 | 6 | 6 |
C:\Users\User\source\repos\lab5\x64\Debug\lab5.exe (процесс 11724) завершил работу с кодом 0 (0x0).
Чтобы автоматически закрывать консоль при остановке отладки, включите параметр "Сервис" ->"Параметры отладки".
Нажмите любую клавишу, чтобы закрыть это окно:
```

Рисунок 9 – результат выполнения проекта при $R=1$, $n=6$.



```
Консоль отладки Microsoft Vi X + v
| Радиус | Кол-во сторон | Периметр |
| 2 | 12 | 12.4233 |
C:\Users\User\source\repos\lab5\x64\Debug\lab5.exe (процесс 25788) завершил работу с кодом 0 (0x0).
Чтобы автоматически закрывать консоль при остановке отладки, включите параметр "Сервис" ->"Параметры отладки".
Нажмите любую клавишу, чтобы закрыть это окно:
```

Рисунок 10 – результат выполнения проекта при $R=2$, $n=12$.

8) Доказательство правильности результатов выполнения проекта

Правильность результатов выполнения проекта подтверждается их совпадением с результатами контрольных вычислений на калькуляторе для обоих вариантов исходных данных.